

Fast answer

A TypeScript arrow function is a concise way to define a function. Its basic syntax is:

TypeScript

```
const functionName = (parameters): ReturnType => {  
  // function body  
  return value;  
};
```

Examples

No parameters

TypeScript

```
const greet = (): string => {  
  return "Hello!";  
};
```

Or as a one-liner:

TypeScript

```
const greet = (): string => "Hello!";
```

One parameter

TypeScript

```
const square = (x: number): number => x * x;
```

Multiple parameters

TypeScript

```
const add = (a: number, b: number): number => {  
  return a + b;  
};
```

Returning an object

Wrap the object in parentheses:

TypeScript

```
const createUser = (name: string, age: number) => ({
  name,
  age,
});
```

Function type annotations

You can explicitly type the variable that holds the function:

TypeScript

```
const multiply: (a: number, b: number) => number =
  (a, b) => a * b;
```

Or use a type alias:

TypeScript

```
type Calculator = (a: number, b: number) => number;

const divide: Calculator = (a, b) => a / b;
```

Generic arrow function

TypeScript

```
const identity = <T>(value: T): T => value;
```

Async arrow function

TypeScript

```
const fetchData = async (): Promise<string> => {
  return "data";
};
```

Why use arrow functions?

- Shorter syntax.
- Lexically binds `this` (inherits `this` from the surrounding scope).
- Commonly used for callbacks.

Example:

TypeScript

```
const numbers = [1, 2, 3];

const doubled = numbers.map(n => n * 2);

console.log(doubled); // [2, 4, 6]
```

If you have a specific use case (e.g., arrow functions in classes, React, generics, or callbacks), I can provide examples tailored to that scenario.